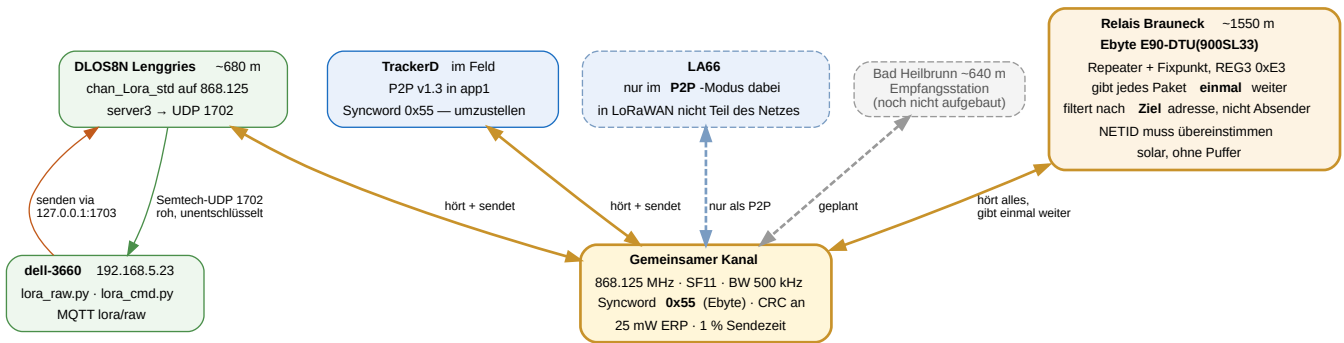


Relaisstelle Brauneck

Krisennetz Lenggries — Aufbau, Betrieb und Befehle. Stand 17.08.2026 · Repository `gerontec/TTN`



1 Grundgedanke: ein gemeinsamer Kanal

Alle Teilnehmer arbeiten auf **einer** Frequenz mit **einem** Spreizfaktor. Das ist keine Sparmaßnahme, sondern zwingend: Ein einzelnes Funkmodul kann immer nur auf einem Kanal lauschen. Nur so hört das Relais jeden und erreicht jeden. Es ist dasselbe Flutungsverfahren, das Meshtastic und Ebytes Broadcast-Modus benutzen.

Parameter	Wert	Warum
Frequenz	868.125 MHz	Werksvorgabe der TrackerD-P2P-Firmware; liegt im Band 868.0–868.6
Spreizfaktor	SF7	reicht: 30 dB Reserve auf 10 km Sichtverbindung
Bandbreite	125 kHz	Standard
Syncword	0x34 (öffentlich)	Historisch, weil der Rohkanal anfangs das Syncword der LoRaWAN-Kanäle mitbenutzen musste. Seit 17.08.2026 nicht mehr nötig — siehe Kapitel 8.
CRC	an	sonst verwirft der Paket-Forwarder
Leistung	bis 14 dBm ERP	25 mW ist das Limit in 868.0–868.6
Sendezeit	1 %	bei ~72 ms Luftzeit rund 7 s Sperre je Paket

Streckenrechnung 10 km, Sichtverbindung. 14 dBm + 2 dBi = 16 dBm EIRP, minus 111 dB Freiraumdämpfung, plus 2 dBi Empfangsantenne ergibt –93 dBm. Gegen –123 dBm Empfindlichkeit bei SF7 bleiben rund **30 dB Reserve**. Den Sprung ins Nachbartal trägt der Bergstandort (~1550 m gegen zwei Täler auf ~650 m), nicht die Sendeleistung.

2 Was das Relais tut

Es hört alles auf dem gemeinsamen Kanal und gibt jedes Paket **genau einmal** weiter, versehen mit einem Sprungzähler. Damit erreicht ein Knoten, dessen Direktsignal zu schwach wäre, das Tal trotzdem.

Nutzlast beginnt mit	Bedeutung	Verhalten
C>	Fernwirkbefehl	ausgeführt, Antwort gesendet, nie weitergegeben
A>	Antwort einer Station	ignoriert
R<n>>	bereits n-mal weitergegeben	weiter bis 3 Sprünge, danach verworfen
alles Übrige	frisches Paket	als R1>... weitergegeben

Eigenecho

Da alle denselben Kanal teilen, trägt allein die Marker-Logik:

- Während des Sendens ist der Empfänger taub — die eigene Aussendung hört die Station nie unmittelbar.
- Jedes weitergegebene Paket bekommt R<sprung>>. Kommt es über eine zweite Relaisstelle zurück, greift der Zähler.
- Der Dublettenspeicher schlüsselt auf den Inhalt **ohne** Marker und sperrt denselben Text fünf Minuten.

Mehrere Relaisstellen sind dadurch möglich: aus R1> wird R2> und so fort, bis drei Sprünge erreicht sind.

3 Fernwirken von 192.168.5.23

Der Pico hat **kein WLAN** — es ist ein RP2040, kein Pico W; das Modul `network` existiert nicht. Auf dem Berg gibt es weder SSH noch OTA. Der einzige Rückkanal ist der Funk, auf dem das Relais ohnehin arbeitet. Für ein Krisensystem ist das der richtige Weg: Er trägt genau dann, wenn alles andere ausgefallen ist.

```
python3 /home/gh/python/lora_cmd.py POWER 17
gesendet: C>POWER 17
Antwort: POWER 17 dBm (RSSI -101, SNR 8.8)
```

Befehl	Wirkung
POWER 2..22	Sendeleistung in dBm — der wichtigste Befehl
STATUS	weitergegeben / unterdrückt / Laufzeit / Konfiguration
RELAY 0 1	Weitergabe aus- oder einschalten
TELEM 0 1	Quittungen aus- oder einschalten
SAVE	Konfiguration ausdrücklich sichern
REBOOT	Neustart des Boards
PING	lebt die Station?

Frequenz und Spreizfaktor sind bewusst nicht fernstellbar. In einem Einkanalnetz würde man sich damit den Ast absägen, auf dem man sitzt — die Station wäre nach dem Wechsel unerreichbar.

Ohne Authentisierung. Wer in Funkreichweite ist, kann das Relais umstellen. Bewusste Abwägung zugunsten der Einfachheit.

Weg des Befehls: `lora-raw.service` hält UDP 1702 dauerhaft und ist der einzige, der senden kann. Er hat deshalb einen Steuereingang auf `127.0.0.1:1703` ; was dort ankommt, wird beim nächsten `PULL_DATA` des Gateways gefunkt. `lora_cmd.py` schickt den Befehl dorthin und wartet über MQTT `lora/raw` auf die Antwort.

4 Betriebsart umschalten

TrackerD — LoRaWAN ↔ P2P

Der TrackerD hat zwei Firmware-Slots: **app0** trägt die Dragino-LoRaWAN-Firmware, **app1** die selbst gebaute P2P-Firmware. Umgeschaltet wird ausschließlich über `otadata` — `app0`, `app1` und vor allem das NVS mit den LoRaWAN-Keys bleiben unangetastet. Der Wechsel ist daher verlustfrei und beliebig wiederholbar.

Richtung	Befehl	gemessen
LoRaWAN → P2P	<code>switch_app.py p2p --port /dev/ttyACM1</code>	1,35 s
P2P → LoRaWAN	<code>AT+LORAWAN</code> — am Gerät, ohne PC-Werkzeug	2,24 s inkl. Neustart
P2P → LoRaWAN	<code>switch_app.py lorawan --port /dev/ttyACM1</code>	gleichwertig
Zustand abfragen	<code>switch_app.py status --port /dev/ttyACM1</code>	liest <code>otadata</code>
P2P neu bauen	<code>pio run</code> , dann <code>switch_app.py flash</code>	schreibt nur <code>app1</code>

Niemals `pio run -t upload`. Das würde Partitionstabelle und `app0` überschreiben und die LoRaWAN-Firmware samt Keys zerstören. Geflasht wird gezielt nach `0x1F0000` durch `switch_app.py`.

LA66 — LoRaWAN ↔ P2P

```
python3 la66_mode.py status      # Modus am Boot-Banner erkennen
python3 la66_mode.py p2p         # 37640 B, 5,0 s
python3 la66_mode.py lorawan     # 69032 B, 9,1 s
```

Beim LA66 wird wirklich geflasht, nicht umgeschaltet: Beide Dragino-Images sind fest auf `0x0800D000` gelinkt, ein zweiter Slot ist unmöglich. Ein Flash setzt die Funkparameter auf Werk zurück — dafür `--apply-rf`. Die DevEUI überlebt.

Im LoRaWAN-Modus ist ein Gerät nicht Teil dieses Netzes. Nachgemessen: Der LA66 sendet mit `AT+DR=0` auf SF12 und springt über alle acht EU868-Kanäle (beobachtet: 867.3 und 868.3 MHz). Der Pico auf 868.125/SF7 hörte in 95 s **null** Pakete. Spreizfaktoren sind quasi-orthogonal — ein SF7-Empfänger kann SF12 nicht demodulieren.

Daraus folgt grundsätzlich: **Ein Knoten mit einem Funkmodul kann kein LoRaWAN-Relais sein.** LoRaWAN wechselt pro Sendung den Kanal und passt per ADR den Spreizfaktor an; ein Empfänger mit fester Frequenz und festem SF kann dem nicht folgen. Genau dafür hat das Gateway acht parallele Demodulatoren. Hinzu kommt: Der Marker `R1>` würde die MIC-Prüfung eines LoRaWAN-Rahmens zerstören.

5 Stromversorgung: reiner Solarbetrieb

Der Victron-Laderegler schaltet den Verbraucher morgens schlagartig zu und abends ab. Eine Pufferbatterie gibt es nicht. Das prägt den Betrieb stärker als jede Funkeinstellung.

- **Jeder Morgen ist ein Kaltstart, und niemand ist oben.** `main.py` startet das Relais, wiederholt bei Fehlern und löst notfalls `machine.reset()` aus, statt in den REPL zu fallen.
- **Konfiguration sichert sich sofort**, nicht erst auf `SAVE` — eine per Funk gesetzte Sendeleistung wäre sonst am nächsten Morgen weg.
- **Beschädigte `relais.json` wird abgefangen**; die Station kommt dann mit Vorgabewerten hoch, aber sie kommt hoch.
- **Nachts ist die Kette unterbrochen.** Eigenschaft des Aufbaus, keine Störung — wer nachts Reichweite braucht, braucht einen Akku.

Systemtakt 48 MHz

125 MHz sind sinnlos: Modulation, Timing und Preamble macht der SX1262 selbst, der Prozessor wartet nur. Weniger Takt heißt weniger Strom, und bei einer Versorgung ohne Puffer weniger Spannungseinbruch unter Last.

Takt	Ergebnis der Abtastung
125 / 96 / 64 / 48 / 32 / 24 MHz	sauber — <code>DevErr 0x0000</code> , Syncword <code>3444</code> , TX ok
18 MHz	SPI liefert Müll — Syncword liest <code>a2a2</code> , TX schlägt fehl
12 MHz	<code>machine.freq()</code> lehnt ab

Gewählt sind **48 MHz**: reichlich Abstand zur Ausfallgrenze bei gut halbiertem Prozessorstrom. Der Takt wird vor dem Aufsetzen des Funkchips gestellt, weil `clk_peri` am Systemtakt hängt.

6 Nachgewiesen über die Luft

Alle drei Stationen in einem Durchlauf, nach dem Kaltstart der neuen Firmware und **ohne** manuelles Nachsetzen des Syncwords:

```
TrackerD sendet  "V13-KALTSTART-1"
Pico            weiter: Sprung 1 RSSI -17 SNR 12.0 20 dBm, 51 ms
Gateway hört    RSSI -83 "V13-KALTSTART-1"      <- direkt
Gateway hört    RSSI -84 "R1>V13-KALTSTART-1"    <- über das Relais
Pico-Bilanz     2 gehört, 2 weitergegeben, 0 unterdrückt
```

In einem früheren Lauf war der Gewinn deutlicher messbar: Original -92 dBm, Kopie vom Relais -73 dBm — **19 dB stärker**. Genau dafür steht die Station auf dem Berg. Der TrackerD empfing dabei sein eigenes Paket als `R1>...` zurück; das Relais gab es **nicht** erneut weiter.

7 Dateien

Ort	Datei	Zweck
Pico	<code>main.py</code>	Selbststart nach jedem Sonnenaufgang
	<code>repeater.py</code>	Flutung, Sprungzähler, Dubletten, Sendezeitbudget
	<code>fernwirk.py</code>	Befehle, Konfiguration in <code>/relais.json</code>
	<code>lora_p2p.py</code>	SX1262-Treiber
dell 192.168.5.23	<code>lora_raw.py</code>	Roh-Abgriff UDP 1702, Steuereingang 1703, MQTT
	<code>lora_cmd.py</code>	Fernwirkbefehle absetzen
TrackerD	<code>p2p/src/main.cpp</code>	P2P-Firmware v1.3, Syncword 0x34 ab Werk
	<code>switch_app.py</code>	otadata-Umschaltung, Flashen nach app1

8 Anleitung: Ebyte-Rohkanal am eigenen Gateway

Diese Anleitung richtet sich an andere Besitzer eines Gateways mit **SX1302** (Dragino DLOS8N, LPS8v2, RAK7268 und Verwandte), die ein Ebyte-Modul — E22, E90-DTU — neben dem laufenden LoRaWAN-Betrieb empfangen wollen. Alle Werte stammen aus eigenen Messungen, nicht aus Datenblättern.

Der Kern in einem Satz. Der SX1302 hat **vier** RX-Syncword-Registerpaare, nicht eines: drei für den gemeinsamen MultiSF-Block (getrennt nach SF5, SF6 und SF7–12, gültig für alle acht LoRaWAN-Kanäle) und **ein eigenes** für den LoRa-Service-Modem hinter `chan_Lora_std`. Der Rohkanal kann deshalb ein beliebiges Syncword tragen, *während* die acht LoRaWAN-Kanäle unverändert auf 0x34 bleiben. Nur schreibt der Semtech-HAL dort stur 0x12 oder 0x34, abgeleitet aus `lorawan_public`.

8.1 Was nicht geht

Einer der acht MultiSF-Kanäle kann **kein** eigenes Syncword bekommen. Die acht teilen sich einen Demodulator-Block, dessen Syncword nur nach Spreizfaktor aufgeteilt ist; ein Register pro IF-Kanal existiert im Chip nicht. Auch Zeitmultiplex hilft nicht: Umschalten kostet zwar nur 590 µs hin und zurück, aber es gibt keinen Auslöser — die Paketerkennung *ist* der Syncword-Vergleich, und wenn ein Paket auffällt, ist es längst verworfen.

8.2 Ebyte-Modul auslesen

Das Modul muss im Konfigurationsmodus stehen ($M0 = 1$, $M1 = 1$; bei USB-Adaptern oft über DTR/RTS). Dann liefert `C1 00 09` die neun Konfigurationsbytes:

```
C1 00 09 | FF FF 00 62 00 12 80 00 00
          ADDH ADDL NETID REG0 REG1 REG2 REG3 CRYPT_H CRYPT_L
```

Feld	Beispiel	Bedeutung
REG2	0x12 = 18	Kanal → Frequenz = 850.125 MHz + Kanal × 1 MHz = 868.125 MHz (900-MHz-Reihe)
REG0 Bits 2–0	2	Lufrate, hier 2.4 k
REG0 Bits 7–5	3	UART-Baudrate, hier 9600
REG3 Bit 6	0	0 = transparent, 1 = Festpunkt mit Adresskopf

Die Lufraten-Etiketten sind nominal — nicht rechnen, messen. Die im Netz kursierende Tabelle übersetzt sie nach BW 125 kHz. Das ist falsch. Gemessen ist die Ebyte-Leiter durchgehend **BW 500 kHz**: Index 2 („2.4k“) ist **SF11/BW500**, Index 5 („19.2k“) ist SF7/BW500. Das Handbuch bestätigt es beiläufig mit „air data rate 2.4kbps@SF11“ — 2,4 kbps bei SF11 gehen rechnerisch nur mit BW 500 auf, bei BW 125 wären es 537 bps.

8.3 Rohkanal am Gateway einstellen

`chan_Lora_std` ist der neunte, eigenständige Kanal. Er wird frei in Frequenz, Spreizfaktor und Bandbreite gesetzt:

```
"chan_Lora_std": {"enable": true, "radio": 1, "if": -375000,
                  "bandwidth": 500000, "spread_factor": 11, ...}
```

Die Frequenz ergibt sich als `radio[n].freq + if`; für 868.125 MHz bei `radio1_freq = 868.5 MHz` also `if = -375000`.

Falle: die Datei, die man editiert, ist die falsche. `init_board()` in `/etc/init.d/lora_gw` ruft bei jedem Start `/usr/bin/generate-config.sh`, und das **kopiert** `/etc/lora/cfg-302/EU-global_conf.json` über `/etc/lora/global_conf.json`. Änderungen an der zweiten Datei sind nach dem nächsten Neustart spurlos weg. Zu ändern ist die **Vorlage**.

8.4 Syncword freischalten

Das Werkzeug liegt im Repository unter `gateway/sx1302_syncword/`. Es ersetzt per `LD_PRELOAD` genau eine HAL-Funktion, `sx1302_lora_syncword()`, deren Signatur keine Structs enthält und daher ABI-neutral ist. Die HAL selbst wird nicht ausgetauscht — bei Dragino steckt der Chip-Reset in `libsx1302hal.so`, ein Upstream-Build würde ihn verlieren.

```
# Cross-Build, OpenWrt-18.06-SDK, mips_24kc, musl
make SDK=/pfad/zu/openwrt-sdk-18.06.9-ar71xx-generic_gcc-7.3.0_musl.Linux-x86_64
make install GW=root@<gateway>
```

`/etc/lora/syncword.conf`:

```
sf5      = auto      # auto = unverändertes Verhalten aus lorawan_public
sf6      = auto
sf7to12  = auto      # 0x34 -> LoRaWAN bleibt unberührt
service  = 0x55      # chan_Lora_std allein: Ebyte-Werkswert
ldro     = 1         # bei BW500 zwingend, siehe unten
```

Aktiviert wird der Shim über den Wrapper `/usr/bin/fwd_syncword`, **nicht** über `procd_set_param env`: `procd` setzt für die Zeilenpufferung selbst `LD_PRELOAD=/lib/libsetlbf.so` und würde einen so gesetzten Wert überschreiben. Der Wrapper hängt den Shim mit `:` getrennt *nach* `procd` an.

LDRO muss von Hand auf 1. Der HAL leitet es aus `SET_PPM_0N(bw, dr)` ab, das nur bei BW125 mit SF11/SF12 und BW250 mit SF12 wahr wird — bei **BW500 also nie**. Ebyte sendet aber mit LDRO 1. Bei falschem LDRO rastet der Header sauber ein, und *jede* Nutzlast kommt mit CRC-Fehler an. Das sieht aus wie „fast richtig“ und ist die zeitraubendste Falle der ganzen Strecke.

8.5 Das Ebyte-Syncword ist 0x55

Ebyte lässt das Syncword nicht konfigurieren; es ist ab Werk verdrahtet und steht in keinem Handbuch. An einem SX126x-Empfänger lässt es sich **nicht vollständig** bestimmen: Der wertet nur das erste der beiden Syncword-Registerbytes aus, weshalb dort alle acht Werte 0x58–0x5F treffen.

Der SX1302 prüft **beide** Peak-Positionen streng und klärt es deshalb. Ein Syncword `0xHL` steckt in den zwei Sync-Symbolen der Präambel; der SX1302 speichert je Symbol `symbolwert / 4`, also **peak_pos = nibble × 2**. Das Feld ist 5 Bit breit und wird vorzeichenlos maskiert — alle 256 Werte sind erreichbar, die Beschränkung auf 0x12/0x34 ist reine Softwarekonvention.

Register	Adresse	gilt für
SF5_PEAk1/2	0x588A/0x588B	alle 8 MultiSF-Kanäle, nur SF5
SF6_PEAk1/2	0x588C/0x588D	alle 8 MultiSF-Kanäle, nur SF6
SF7T012_PEAk1/2	0x588E/0x588F	alle 8 MultiSF-Kanäle, SF7–12
LORA_SERVICE_PEAk1/2	0x5B2E/0x5B2F	nur chan_Lora_std

8.6 Wenn nichts ankommt: peak2 absuchen

Fremde Module können ein anderes unteres Nibble benutzen. Statt den Forwarder sechzehnmal neu zu starten, wird das Register im laufenden Betrieb gesetzt — der SX1302 hat kein Page-Register, jeder Zugriff ist ein einzelnes `ioctl` und wird vom Kernel gegen den Forwarder serialisiert:

```
for n in $(seq 0 15); do
    sx1302_poke 0x5B2F $((n*2))
    vorher=$(grep -c '"chan":8' /tmp/fwd.log); sleep 3
    nachher=$(grep -c '"chan":8' /tmp/fwd.log)
    printf 'sw=0x5%X peak2=%2d neue Pakete: %d\n' $n $((n*2)) $((nachher-vorher))
done
```

Das Sendegerät muss dabei durchgehend funken. Genau so wurde 0x55 gefunden: Pakete ausschließlich bei `peak2 = 10`.

8.7 Prüfen

```
logread | grep syncword
[syncword] LoRa Service LDR0 forced to 1 (HAL rule overridden)
[syncword] multi-SF ch0-7: SF5=0x12 SF6=0x12 SF7-12=0x34 | Lora_std (SF11): 0x55

sx1302_poke 0x5B2E -> 0x0A peak1, oberes Nibble 5
sx1302_poke 0x5B2F -> 0x0A peak2, unteres Nibble 5
sx1302_poke 0x5B22 -> 0x98 Bits 4-5 = 1, LDR0 aktiv
```

Ein empfangenes Paket erscheint als `rxpk` auf **chan 8**:

```
{"chan":8,"freq":868.125000,"datr":"SF11BW500","rssi":-63,"stat":1,
 "size":15,"data":"LBKHJgD//wdCQF1WPyIh"}
```

8.8 Das Ebyte-Rahmenformat

Auch im transparenten Modus verpackt das Modul die Nutzlast. Der Kopf ist acht Byte lang, und die Nutzlast ist mit der **Kanalnummer** XOR-verweift — dieselbe Zahl steht in Byte 1:

```
2C 12 87 26 00 FF FF 07 | 42 40 5D 56 3F 22 21
  ^^                               XOR 0x12
Kanal 18 = zugleich der Schlüssel      -> "PROD-03"
```

```
Byte 0      0x2C    Kennung
Byte 1      Kanal   zugleich XOR-Schlüssel
Byte 2-3     laufende Nummer
Byte 4      NETID
Byte 5-6     Adresse des Absenders (FF FF = Broadcast)
Byte 7      Länge der Nutzlast
```

8.9 Was der Eingriff nicht anfasst

Nachgemessen mit einem LA66-USB (EU868 v1.3, bereits gejoint): Uplink auf 868.500 MHz bei DR0 kommt unverändert auf `chan 2 an, SF12BW125, stat:1`, Rahmen sauber — MHDR 0x40, DevAddr, FPort 2, Nutzlast lesbar. **Der LoRaWAN-Betrieb bleibt vollständig erhalten**, weil ausschließlich das Register des Service-Modems verändert wird.

8.10 Alles in einem Skript

Die fünf Handgriffe — Vorlage, `syncword.conf`, Wrapper, Init-Skript, Neustart — fasst `gateway/sx1302_syncword/setup_ebyte_rawchannel.sh` zusammen. Es läuft auf dem Gateway, sichert jede angefasste Datei nach `.pre-ebyte` und prüft sich am Ende selbst:

```
setup_ebyte_rawchannel.sh --apply      # einrichten und pruefen
setup_ebyte_rawchannel.sh --status     # Ist-Zustand inkl. Registern
setup_ebyte_rawchannel.sh --revert     # alles zurueck, Stock-Verhalten
```

Vorgaben passen zu einem Ebyte auf Werkskonfiguration; abweichend etwa `--sf 7 --bw 500000 --sync 0x55` für Luftrate 19.2k. Ausgabe eines erfolgreichen Laufs:

```
==> Vorlage angepasst (Sicherung: ../EU-global_conf.json.pre-ebyte)
==> Init-Skript angepasst (Sicherung: /etc/init.d/lora_gw.pre-ebyte)
[syncword] LoRa Service LDR0 forced to 1 (HAL rule overridden)
[syncword] multi-SF ch0-7: SF5=0x12 SF6=0x12 SF7-12=0x34 | Lora_std (SF11): 0x55
0x5B2E = 0x0A (10) peak1
0x5B2F = 0x0A (10) peak2
```

8.11 Die Gegenrichtung: Gateway sendet

Alles oben betrifft den **Empfang**. Soll das Gateway den Ebyte-Knoten auch *erreichen* — etwa einen E90-DTU-Repeater auf dem Berg —, greift eine andere Stelle im HAL: `sx1302_send()` schreibt das Sende-Syncword **für jedes Paket neu**, unmittelbar vor dem Tasten, wieder abgeleitet aus `lorawan_public`. Ein Downlink geht damit stets als 0x34 hinaus, und eine Gegenstelle auf 0x55 hört ihn nie.

Der Shim kann auch das (`tx = 0x55` in `syncword.conf`); umgesetzt ist es durch Interposition von `lgw_com_rmw()` mit Umschreiben der vier TX-Register `0x526D/0x526E` und `0x546D/0x546E` — adressbasiert, also wieder ohne Bindung an Draginos Strukturen.

`tx` allein würde auf *jeden* Downlink wirken, auch auf die von LoRaWAN — Join-Accepts und ADR gingen dann auf einem Syncword hinaus, das kein Endgerät annimmt. Deshalb gibt es `tx_freq`:

```
tx          = 0x55
tx_freq = 868125000    # nur Downlinks auf dieser Frequenz, Fenster +/- 5 kHz
```

Möglich wird das dadurch, dass `sx1302_send()` die Sendefrequenz in `loragw_sx1302.c:2604-2608` schreibt, **bevor** es in `:2710` zum Syncword kommt. Der Shim liest die drei Frequenzbytes im Vorbeigehen mit (sie laufen als 8-Bit-Direktschreibung über `lgw_com_w()`, nicht über `lgw_com_rmw()`) und entscheidet dann je Sendekette. Die Auflösung beträgt $32 \text{ MHz} / 2^{18} = 122 \text{ Hz}$, das trennt den Rohkanal auf 868.125 mühelos von `chan_multiSF_0` auf 868.100, 25 kHz daneben.

tx_freq	Downlink auf 868.125	peak1 / peak2	Syncword
868125000	Rohkanal getroffen	10 / 10	0x55 umgeschrieben
869525000 (RX2)	passt nicht	6 / 8	0x34, HAL-Wert unberührt

Beim Ablesen der Register nicht stolpern: das Peak-Feld sind die **unteren fünf Bit**. `0x526D = 0xAA` heißt `0xAA & 0x1F = 10`; die oberen Bits tragen `AUTO_SCALE`, `GAIN` und `DROP_ON_SYNC`.

Funkrechtlich. 868.0–868.6 MHz erlaubt 25 mW ERP bei 1 % Sendezeit. Ein Rohkanal mit BW 500 kHz auf 868.125 MHz belegt 867.875–868.375 MHz und überlappt damit die LoRaWAN-Kanäle 868.1 und 868.3 — im selben Unterband zulässig, aber beim Sendezeitbudget mitzurechnen.

Erzeugt aus `devices/pico_sx1262/` im Repository `gerontec/TTN`. Zeichnung: `relais_uebersicht.dot`.